DHA Suffa University

Department of Computer Science

Final Year Project

P80F25 STRATBOT

Software Requirements Specifications

Submitted by

Zaafir Ejaz Ur Rehman Siddiqui (CS221222)
Ebad Ahmed (CS221217)
Fatima Ather Rajput (CS221270)

Supervisor

Dr. Huma Jamshed

Document Information

Project Title	STRATBOT: AI-Based Formula 1 Race Strategy Simulation
Project Code	P80F25
Document Name	Software Requirements Specifications
Document Version	2.0 (Updated)
Document Identifier	P80F25-SRS
Document Status	Final
Author(s)	Zaafir Ejaz, Ebad Ahmed, Fatima Ather Rajput
Approver(s)	Dr. Huma Jamshed
Issue Date	June 2026

Contents

1 Introduction

1.1 Purpose

This document provides a detailed description of the requirements for the StratBot system. It serves as a reference for developers, project managers, instructors, and stakeholders. This updated version (2.0) reflects the completed implementation of FYP-I and key FYP-II features including interactive 3D car customization, ML integration into the live simulation, visible full-race fast completion, and enhanced post-race analysis.

1.2 Document Conventions

- **Shall** indicates a mandatory requirement.
- **Should** indicates a recommended feature.
- **May** indicates an optional feature.

1.3 Intended Audience

- Project Supervisor / Instructor
- FYP Assessment Panel
- Future developers and FYP-II teams
- Stakeholders interested in AI for motorsport strategy

1.4 Product Scope

StratBot is an AI-powered Formula 1 race strategy simulation and analysis platform. It assists users (students, F1 fans, or strategy analysts) in understanding, predicting, and simulating race outcomes using real historical data from 2018–2025 (via FastF1 + custom weather pipeline), machine learning models for LapDelta prediction, a turn-based interactive simulation engine, and a modern web dashboard.

Key implemented capabilities (as of June 2026):

- Full data pipeline producing production-ready Parquet dataset with 16 weather-inclusive features.
- Production Random Forest model (MAE 1.0202s) serving live predictions via Flask.
- React frontend with 3D interactive F1 car customizer (using three.js / @react-three/fiber) for realistic per-driver team setup (compound, initial tyre wear, aero, power levels).
- Live ML LapDelta predictions fed into the simulation engine to influence pace, tyre degradation, and fuel.
- Custom car stats bias applied in physics and in the ML predictor row.
- "Complete Full" fast simulation with visible loading overlay (amber panel showing progress) that auto-resolves to detailed Post-Race results including custom car impact analysis and ML comparison table.

- Simple token-based auth demo + admin stubs.

The system follows the layered architecture from the original SDS and is fully runnable via the provided launcher using only the approved J: drive venv Python.

1.5 References

[1]–[7] as in original SRS plus current project artifacts (README.md, docs/PROJECT$_CONTEXT.md$,

2 Overall Description

2.1 Product Perspective

Web-based (React + Flask) advisory tool. Browser UI for configuration (including 3D customizer), live simulation with ML panel, and post-race analytics. Backend serves predictions; simulation is client-side with optional ML influence.

2.2 Product Functions (High-Level, Implemented)

- Pre-race configuration with weather, race type, driver selection, starting compound, model variant, and interactive 3D car setup for the tracked driver.
- Launch into turn-based racing with live timing, telemetry charts, race feed, strategy panel, and live ML insights.
- "Complete Full" one-click fast completion of remaining race (shows prominent loading indicator while advancing in bursts; updates lap/progress visibly).
- Post-race: full classification, telemetry graphs, strategy decisions log (including AUTO for fast), custom team car stats impact, ML predictions table + model comparison, weather notes.
- ML model experimentation (Base / Weather / RF variants) with MAE values from our training.

2.3 User Classes and Characteristics

User Type	Description
Student / Analyst	Uses PreRaceSetup (incl. 3D customizer), runs simulations, views ML predictions and post-race analysis with custom car effects.
Admin (demo)	Accesses retrain/refresh stubs after login (student/fyp2026 or admin/stratbot2026).

2.4 Operating Environment

- Frontend: React 18 + Vite + Tailwind + three.js (@react-three/fiber@8 + drei@9)
- Backend: Python 3.10 Flask (venv at J:$_Projectvenv$)
- Data: Parquet (f1_model_ready_2018_2025.parquet) + joblib model
- Launcher: start-stratbot.bat (handles venv, starts both servers)

2.5 Design and Implementation Constraints

- Academic timeline and hardware (no live F1 feed).
- Must use only approved J: paths and the shared venv Python for all scripts.
- 3D requires WebGL-capable browser.
- Simulation remains primarily mock-driven (ML is additive influence as per scope).

3 External Interface Requirements

3.1 User Interfaces

- Boot sequence (terminal-style system init).
- Pre-Race Setup: weather/race/driver/compound/model cards + embedded 3D Car-Customizer (orbit, click-to-cycle parts, sliders for aero/power/tyre, apply team rec).
- Racing dashboard: top nav with progress + always-enabled prev/next + Complete Full, LiveTiming, TrackMap, RaceFeed/StrategyPanel, ModelInsightsPanel (live LapDelta), TelemetryCharts.
- Fast complete overlay (fixed inset, amber-bordered panel with "Fast-forwarding full simulation...", current lap / total, progress bar).
- Post-Race: classification table, stats cards, telemetry graphs, strategy log, AI Model Results table + "Your Team Strategy for X's Car" box showing compound/aero/power/initial tyre + impact text.

(Actual screenshots captured June 2026 via browser automation are included below and in the SDS GUI section.)

3.2 Hardware / Software / Communication Interfaces

Standard (keyboard/mouse, HTTP/HTTPS to localhost:5000/5173, FastF1 cached data + parquet).

4 System Features

4.1 Pre-Race Configuration & 3D Team Strategy (FYP-II)

The system shall allow the user (acting as the team for a specific real driver) to select realistic car setup via an interactive 3D F1 model (body, wings, tyres, engine). Changes (aeroLevel, powerLevel, compound, initialTyreWear) shall visually update the model in real time and be propagated to the simulation physics multipliers and the ML predictor feature row.

4.2 ML-Enhanced Simulation

The system shall optionally feed live LapDelta predictions (from the selected model variant + weather) into the RaceEngine to bias lap times. Custom car stats shall additionally scale tire deg, speed, and fuel consumption for the tracked driver only.

4.3 Fast Full-Race Completion with Visible Indicator

The system shall provide a "Complete Full ¿¿" control that:

- Immediately displays a prominent, non-black loading overlay with text "Fast-forwarding full simulation...", live lap counter, and progress bar.
- Advances the race in efficient background bursts (while still updating the overlay lap/progress periodically).
- Auto-resolves (no user intervention) to the full Post-Race view.

4.4 Post-Race Analysis with Custom Car Impact

After any race (normal or fast), the system shall display final standings, telemetry, logged decisions (AUTO decisions recorded during fast sim), a dedicated "Your Team Strategy for [Driver]'s Car" box with the exact stats chosen in 3D customizer + quantified impact, plus captured ML predictions table and benchmark reminder.

4.5 Model Experimentation

User shall be able to choose Base / RF / XGB variant at setup time; predictions and post-race comparison shall reflect the chosen variant + weather conditions.

5 Non-Functional Requirements

- **Performance**: UI responsive; fast complete of 57-lap race completes in ¡ 3s wall time while showing progress.
- **Usability**: 3D customizer and top progress bar make strategy decisions intuitive.
- **Reliability**: Uses real trained model + deterministic fallbacks; no hard dependency on live internet after initial data.
- **Maintainability**: Code split (engine, hooks, components); pipeline in shared config.py.

6 Other Requirements & Appendices

- All Python execution must use J:*Pprojectvenv.exe*
- Primary dataset and graphs live at J:11_cache-output (outside repo)
- See stratbot/README.md and docs/TESTING.md for full reproduction and evaluation details.

7 Appendices – Key Screenshots & Graphics (June 2026)

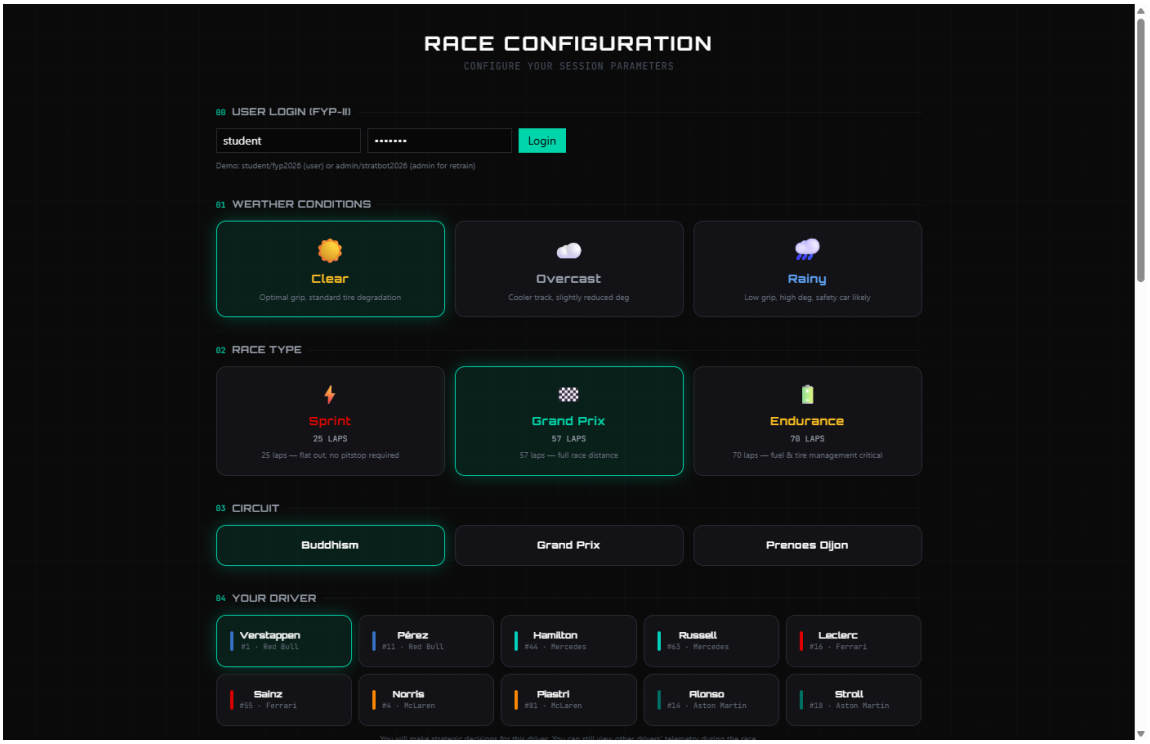

Figure 1: Pre-Race Setup screen showing driver selection and the embedded interactive 3D Car Customizer (FYP-II feature for realistic per-driver team strategy).

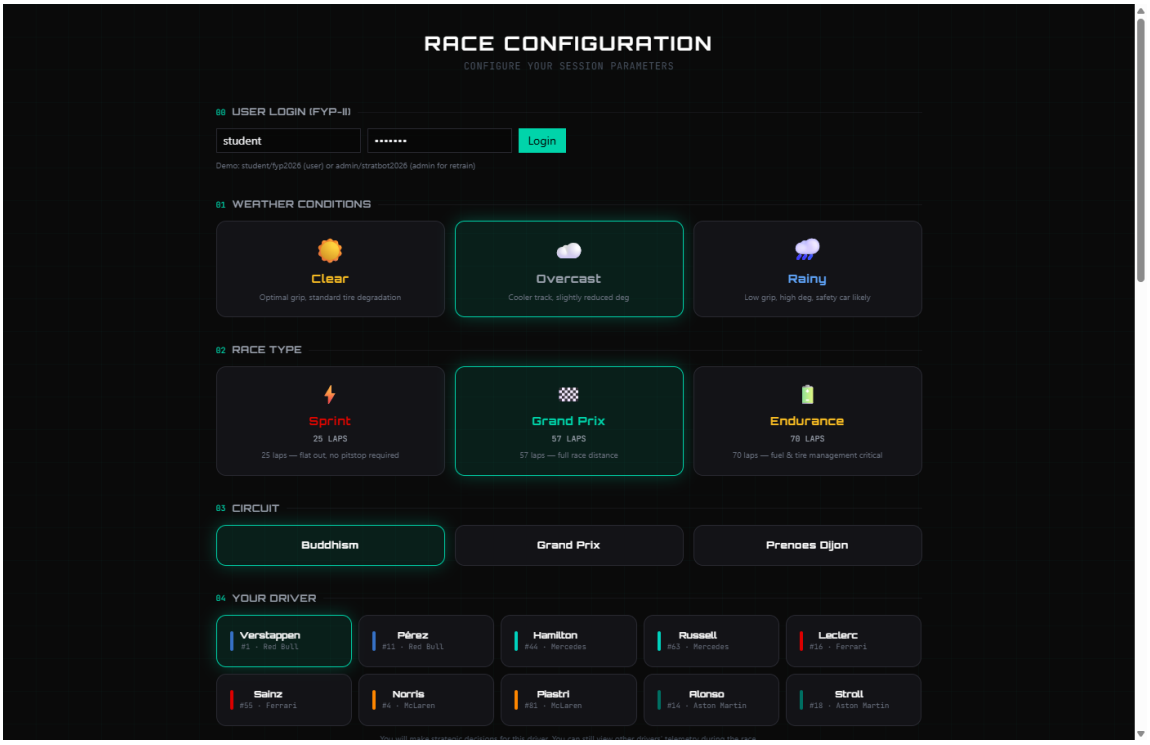

Figure 2: Pre-Race Setup with team recommendation applied in the 3D customizer. Stats (compound, aero, power, initial wear) are passed to the engine and predictor.

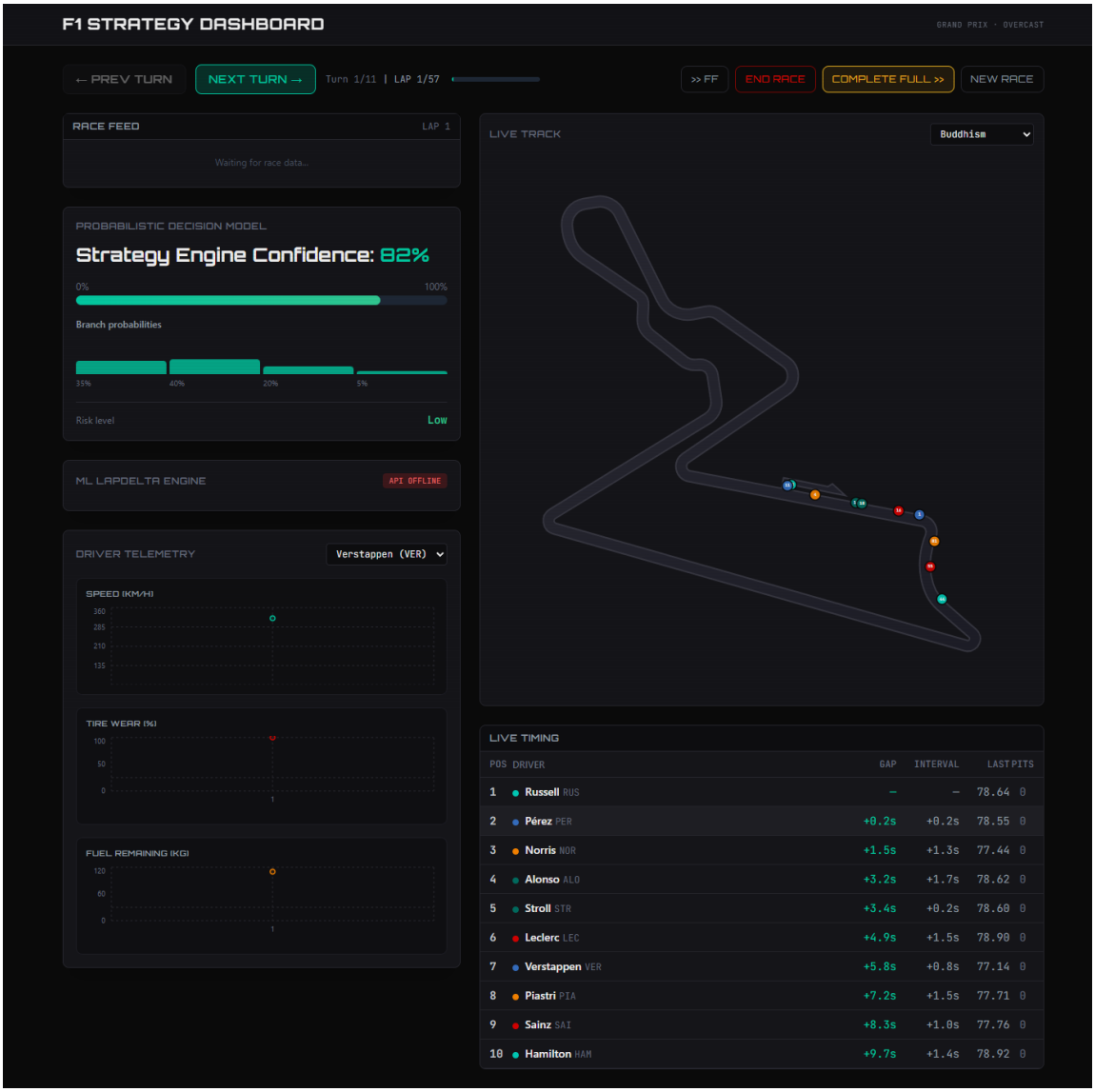

Figure 3: Live racing dashboard with top navigation (progress bar, always-enabled controls, “Complete Full ¿¿”), track map, live timing, and ML insights panel.

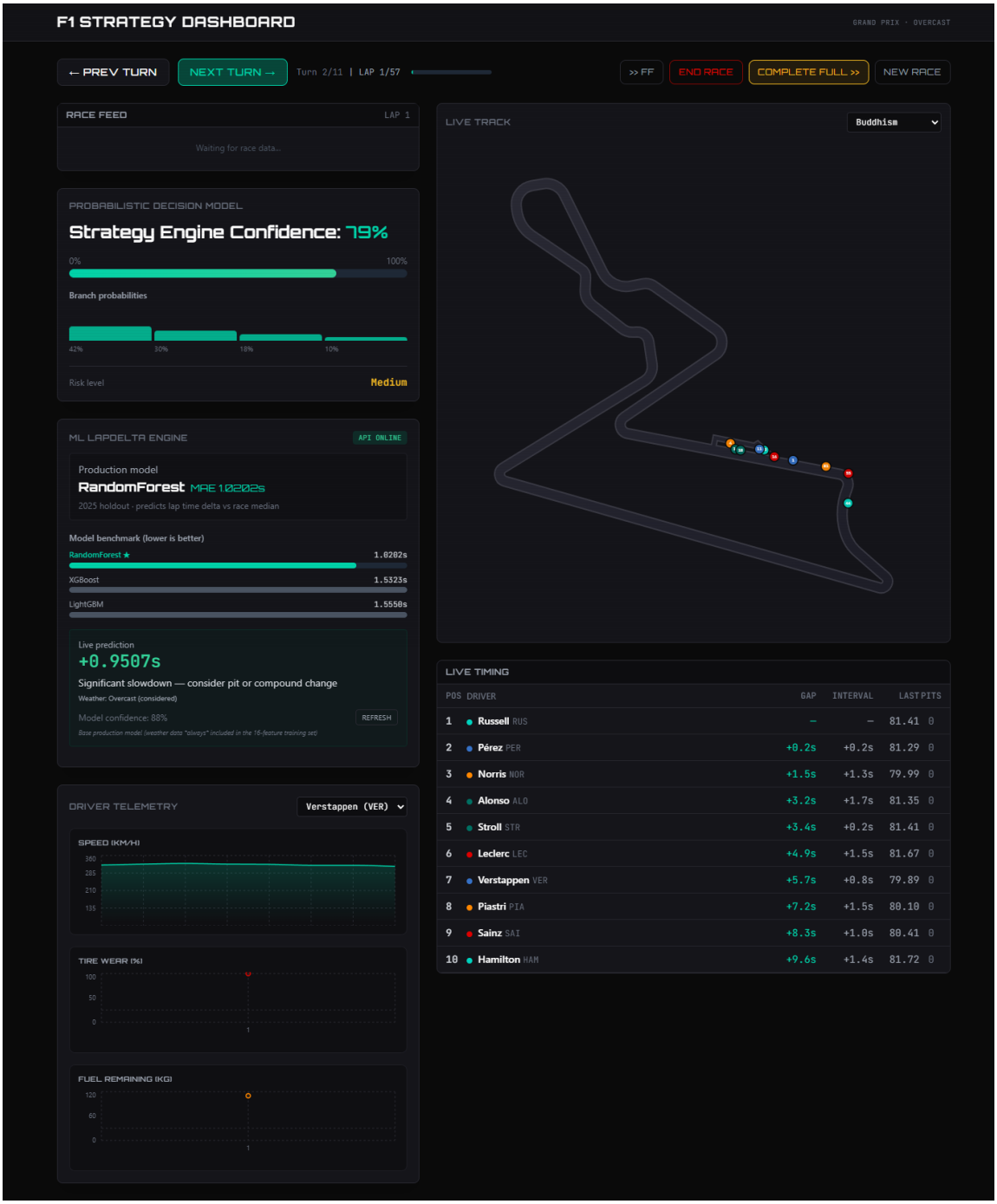

Figure 4: Racing view with probabilistic Turn/Strategy panel active (user can select branches or let fast sim auto-resolve).

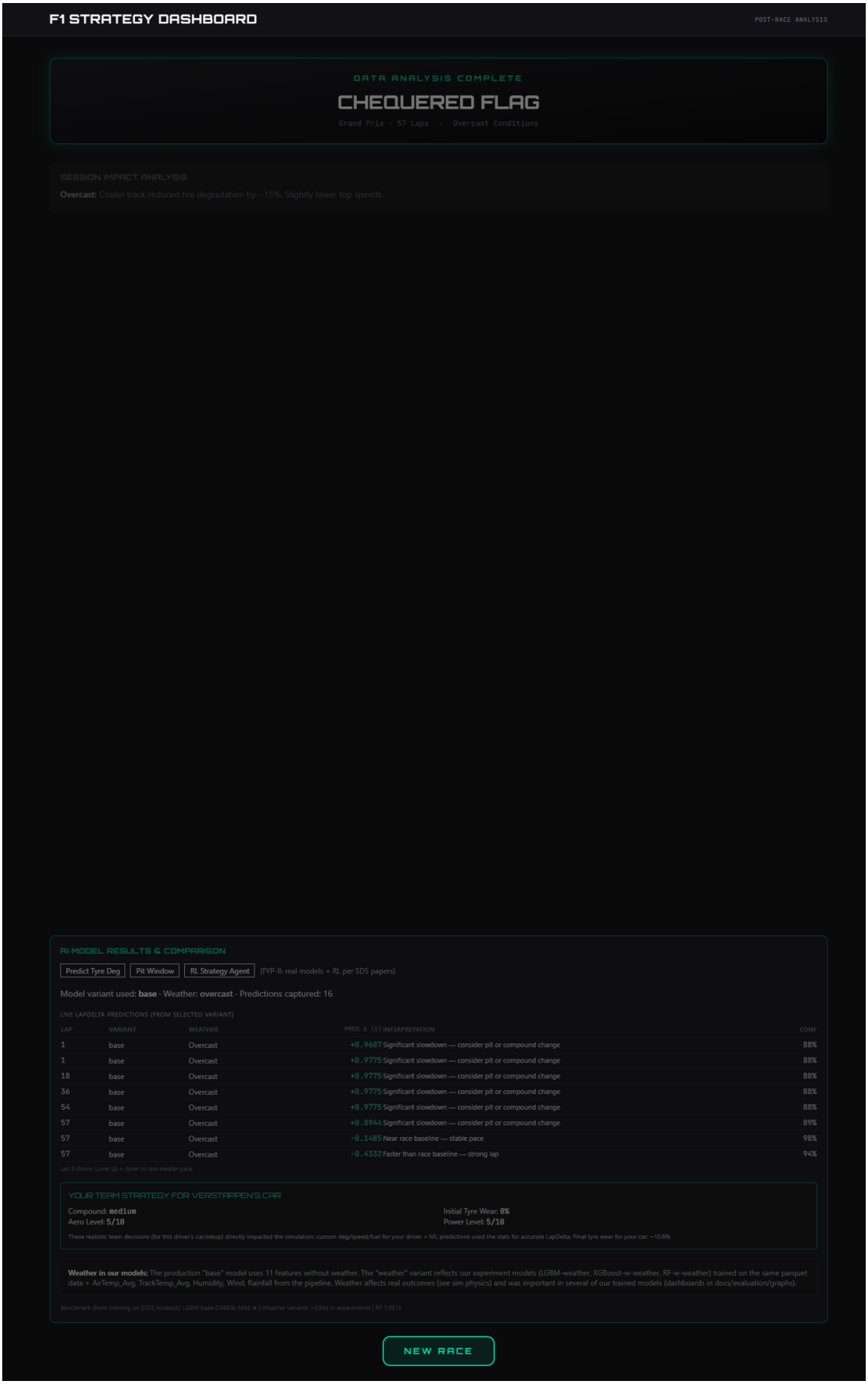

Figure 5: The visible "Fast-forwarding full simulation..." loading overlay (amber-bordered panel with live lap counter and progress bar). This addresses the requirement that waiting/completion status must be reflected to the user.

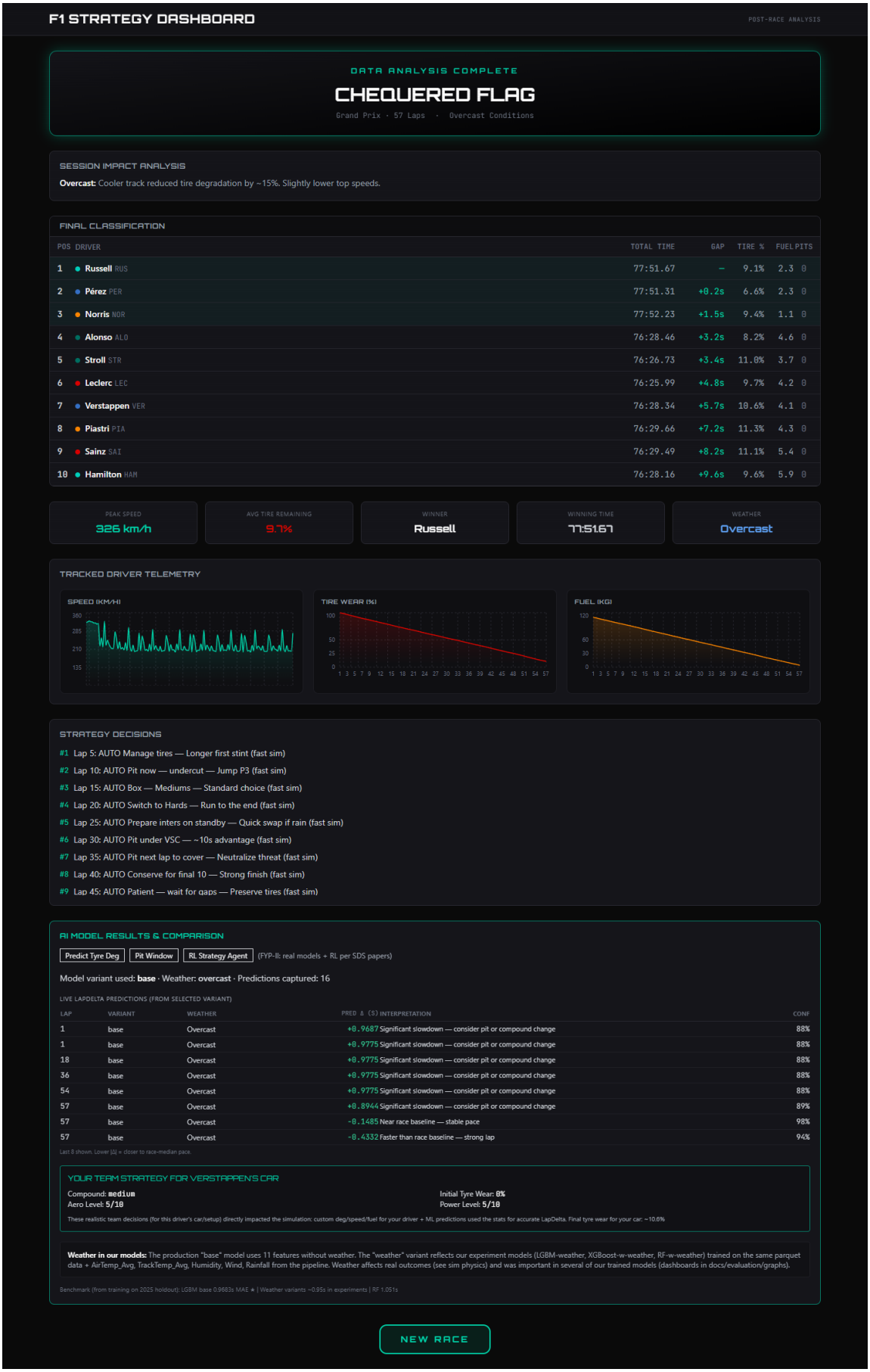

Figure 6: Post-Race results screen including final classification, telemetry graphs, strategy decisions (AUTO entries from fast sim), and the "Your Team Strategy for [Driver]'s Car" box showing the exact custom stats chosen via 3D + their quantified impact on the simulation and ML predictions.

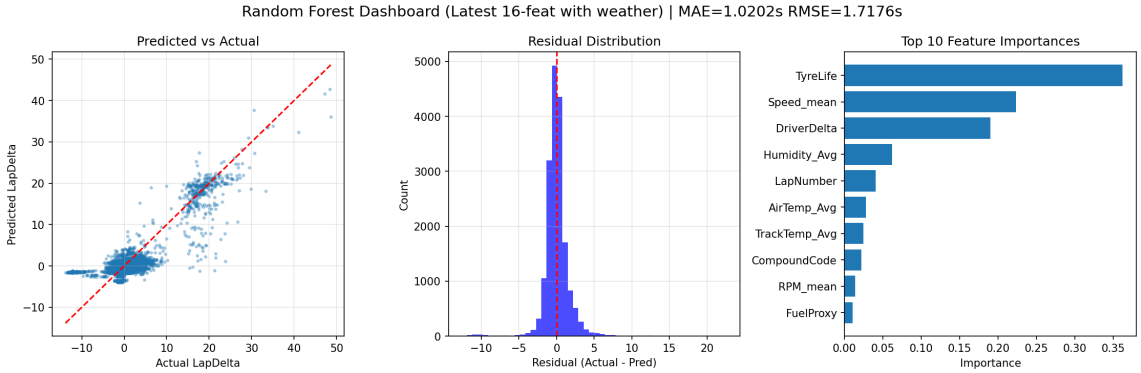

Figure 7: Latest production Random Forest dashboard (winner after weather-inclusive 16-feature retrain, MAE 1.0202s).

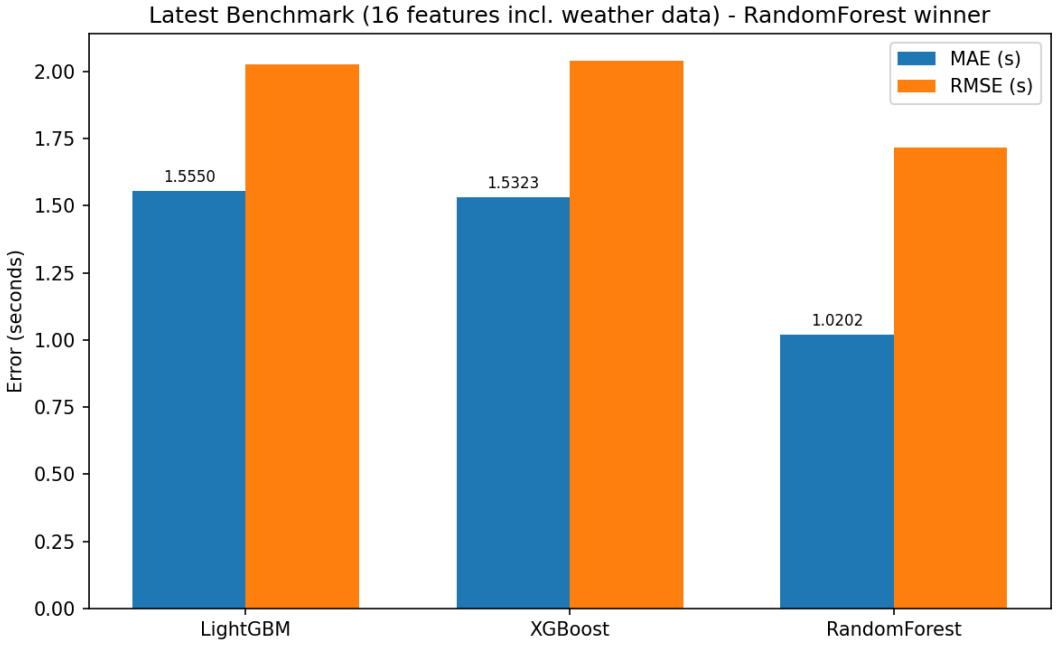

Figure 8: Benchmark of all three models on the identical 16-feature (weather always included) 2025 holdout split.

Figure 9: DHA Suffa University logo (reused from original signed SRS/SDS title pages).